

BBT: BPE-Guided Byte Transformer

Keep BPE segmentation for compression; replace vocabulary lookup and LM head with byte-level composition.

MAIN CLAIM

BBT effectively models the roles of embedding layers and LM heads through byte-level composition.

Hyunho Kim Kakao Corp
khe1.2h@kakaocorp.com

SeungYeol Baek AgileSoDA
sybaek1005@gmail.com



1.1 The problem

A BPE tokenizer does **two different jobs**:

- **Segmentation** — bytes become a shorter unit sequence, cutting global attention cost.
- **Vocabulary lookup** — the same ids index the embedding matrix and define the LM head's prediction space; rarely examined *in isolation*.

Representation. Every token gets its own embedding / LM head row, so structure shared across tokens — prefixes, suffixes, spelling — is invisible to the vocabulary, and rare tokens are learned from their own sparse counts.

Parameters. Both matrices scale with the vocabulary: in the LLaMA 3.2 1B, the 128K-token embedding alone is **262.7M** — **21.3% of the 1.23B total**.

1.2 Our answer

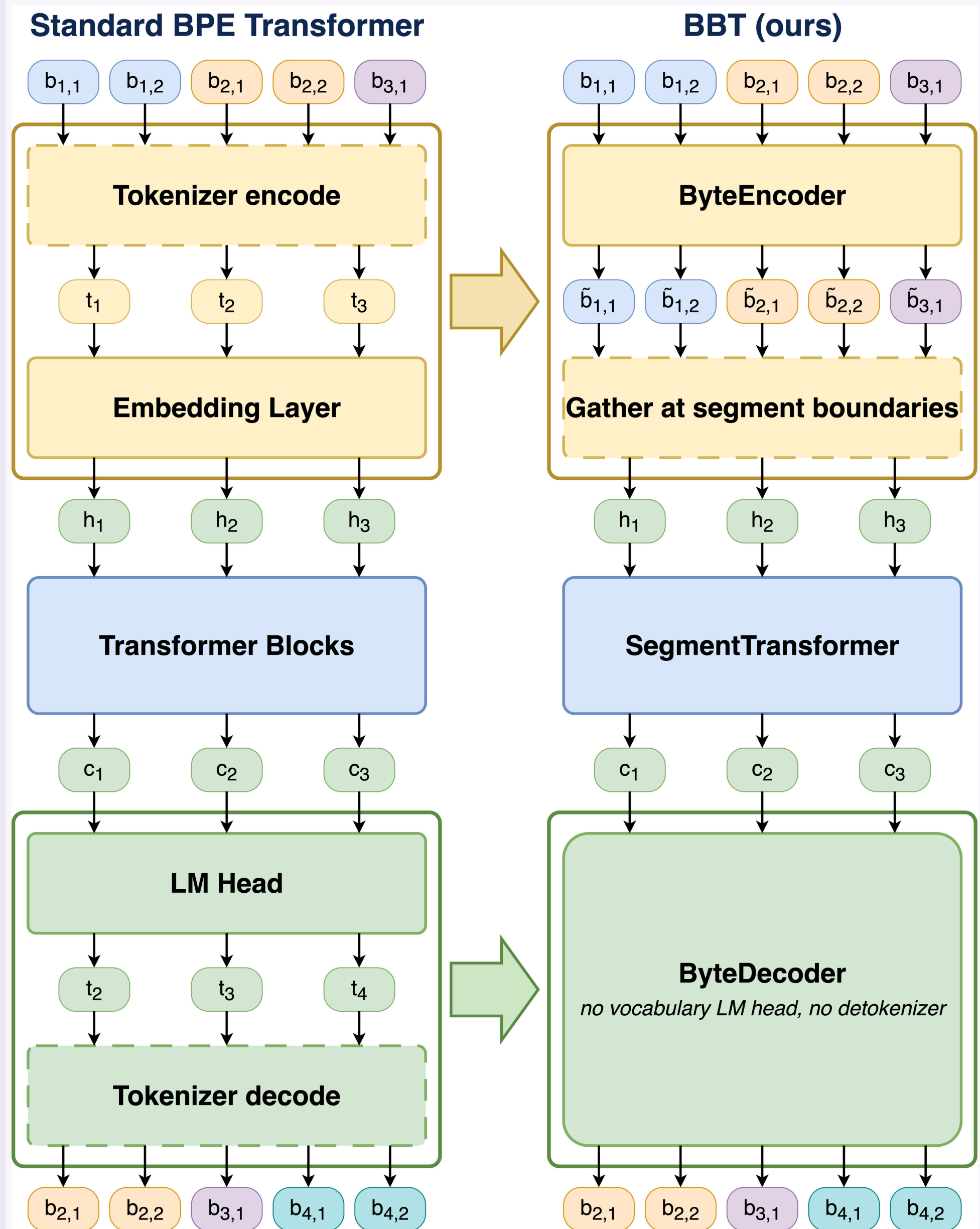


Figure 1. Standard BPE Transformer (left) vs BBT (right) — only the two outlined interface regions differ.

The **BPE-Guided Byte Transformer (BBT)** keeps only one of BPE's two jobs: the merge table still sets the boundaries, but BPE's token ids are **never used at either end**.

The merge table is fit to the training corpus, so those boundaries already encode **corpus statistics**.

The lookup is another matter: what the **embedding and LM head rows** hold can instead be *composed* from the bytes each token is made of.

BBT does exactly that: a **ByteEncoder** turns bytes into **BPE-segment** representations, and a **ByteDecoder** writes each segment back out as **bytes**. Neither side scales with $|V|$.

1.3 Where BBT sits

Byte / character models (ByT5, Charformer, CANINE) drop the vocabulary and gain open-vocabulary robustness, but sequences get ~4–5× longer, raising global attention cost.

Hierarchical byte models (MegaByte, SpaceByte, BLT, ByteFlow) restore compression with local+global computation and are *tokenizer-free*: boundaries come from fixed lengths, surface heuristics, entropy models, or learned chunkers.

BBT takes the middle ground: **BPE-derived boundaries stay fixed** and only the interface changes — so every comparison here attributes the difference to the *interface*, not to boundary placement.

2.1 Method

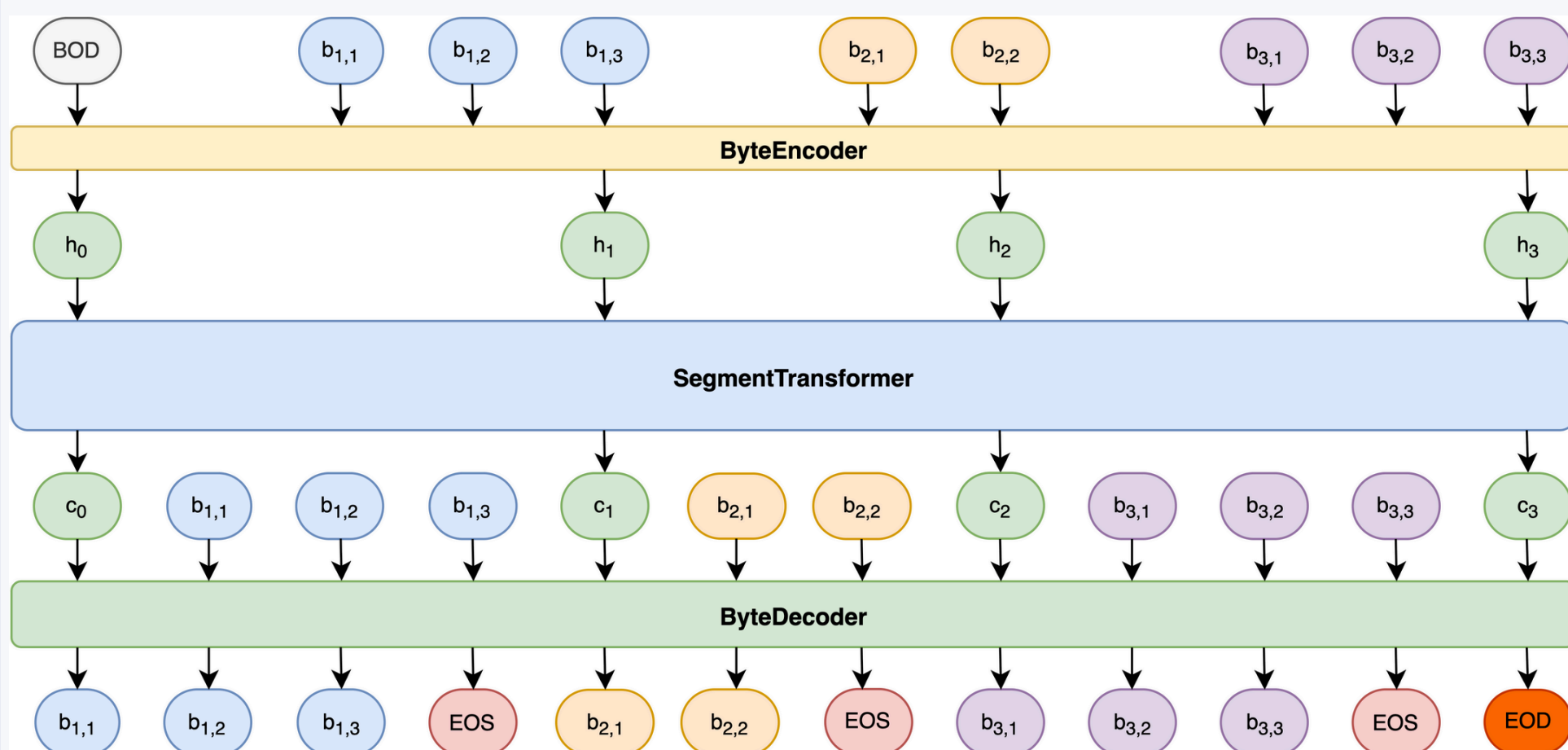


Figure 2. Training architecture. Contexts c_i are interleaved with teacher-forced bytes, so every byte of segment $i+1$ is predicted from c_i .

① ByteEncoder

Each byte is embedded as a vector and run through a sliding-window causal Transformer. The representation h_i of segment i is the state at its **last byte** — the segment plus its left context, never future bytes.

② SegmentTransformer

Causal self-attention over $(\tilde{h}_0, \dots, \tilde{h}_N)$ yields contexts $(c_0, \dots, c_N)$. Attention length scales with the **number of BPE segments N** , not the byte length T — this is where BPE's compression is preserved.

③ ByteDecoder

A sliding-window causal Transformer over the interleaved sequence $[c_0, b_{1,1 \dots L_1}, c_1, b_{2,1 \dots L_2}, \dots, c_N]$. Each c_i predicts the next segment's first byte, byte positions predict the following byte, and a segment's final byte predicts EOS; c_N predicts EOD.

Objective

$$\mathcal{L} = - \sum_{i=0}^{N-1} \sum_{j=1}^{L_{c_i}} \log p(b_{i+1,j} | c_i, b_{1,1 \dots L_1}, \dots, b_{i,1 \dots L_i}) - \lambda_{\text{seg}} \sum_{i=0}^{N-1} \log p(\text{EOS} | c_i, b_{1,1 \dots L_1}, \dots, b_{i,1 \dots L_i}) - \log p(\text{EOD} | c_N)$$

Byte NLL + segment termination + document termination.

2.2 Inference

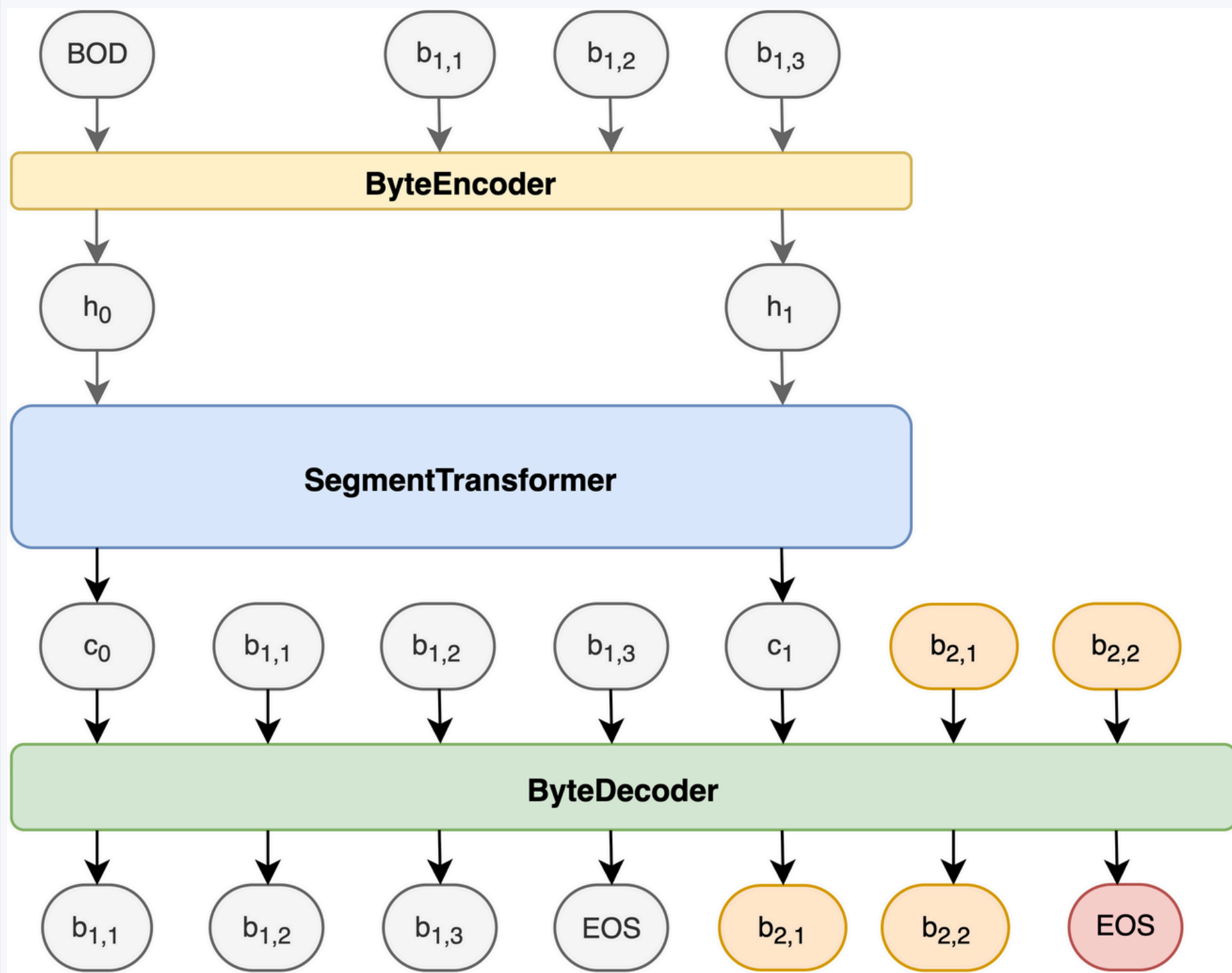


Figure 3. Inference process.

BBT emits bytes, but it never runs the whole stack per byte.

Inside a segment only the **ByteDecoder** extends; the **ByteEncoder** and **SegmentTransformer** stay untouched.

On EOS the finished segment is appended to the ByteEncoder input, and its boundary state becomes the next context.

The two upper modules advance **once per segment, not once per byte** — global attention is never paid at byte rate.

3.1 Setup & main result

Data. FineWeb subset: **47.3 GB** of raw training text ($\approx 10\text{B}$ BPE tokens), 15 MB held out each for val/test. A **128K byte-level BPE tokenizer** is trained on the training split.

Matched pairs. At each hidden size, the standard BPE Transformer body and BBT's SegmentTransformer share hidden size, layers, heads, and FFN dim.

Metric. **BPB** = NLL of the original byte sequence $\div$ its byte count, so both families are scored on the *same* byte stream. $\text{BPB}_{\text{eos-as-byte}}$ also charges the EOS NLL, counting each EOS as one byte-like target.

d	Model	#Params	BPB	$\text{BPB}_{\text{eos-as-byte}}$
1024	Std BPE	266.4M	1.0200	—
	BBT	158.8M (−40.4%)	0.9924 (−2.7%)	0.9666
2048	Std BPE	983.6M	0.9690	—
	BBT	761.8M (−22.5%)	0.9272 (−4.3%)	0.9009
3072	Std BPE	3.605B	0.9447	—
	BBT	2.879B (−20.2%)	0.8823 (−6.6%)	0.8566

BBT wins at every matched size and the gain **grows with scale** (−2.7% $\rightarrow$ −6.6%). The win survives even when EOS is charged as a byte.

3.2 Where the savings come from

d	Component	Parameters			Forward FLOPs/byte		
		Std BPE	BBT	Δ	Std BPE	BBT	Δ
1024	Input side	131.1M	0.4M	−130.7M	0.0M	0.5M	+0.5M
	Segment body	135.3M	135.3M	± 0	61.2M	61.2M	± 0
	Output side	0 (tied)	23.1M	+23.1M	54.2M	56.0M	+1.8M
	Total	266.4M	158.8M	−40.4%	115.4M	117.7M	+1.9%
2048	Input side	262.1M	1.3M	−260.8M	0.0M	1.6M	+1.6M
	Segment body	721.5M	721.5M	± 0	316.6M	316.6M	± 0
	Output side	0 (tied)	39.0M	+39.0M	105.6M	86.0M	−19.6M
	Total	983.6M	761.8M	−22.5%	422.1M	404.2M	−4.2%
3072	Input side	393.2M	4.7M	−388.5M	0.0M	6.3M	+6.3M
	Segment body	2.819B	2.819B	± 0	1.212B	1.212B	± 0
	Output side	393.2M	55.1M	−338.1M	159.5M	120.7M	−38.8M
	Total	3.605B	2.879B	−20.2%	1.372B	1.339B	−2.4%

Input side = token embedding vs. ByteEncoder + projection; output side = vocabulary LM head vs. ByteDecoder + projection.

4.1 Beyond likelihood

	$d=1024$		$d=2048$		$d=3072$	
Metric	Std BPE	BBT	Std BPE	BBT	Std BPE	BBT
LAMBADA accuracy (↑)						
LAMBADA	26.70	26.00	30.02	33.81	30.20	38.17
WikiText robustness						
Clean BPB (↓)	1.1859	1.1700	1.1263	1.0957	1.1013	1.0483
Relative BPB degradation over clean (↓)						
Char drop	114.5%	78.5%	119.3%	81.7%	121.4%	83.6%
Space insert	147.8%	98.1%	151.2%	95.0%	153.7%	89.8%
Space drop	34.5%	31.6%	34.5%	29.6%	34.5%	28.7%
Shuffle	210.8%	167.4%	223.3%	177.2%	228.8%	184.5%

BBT improves completion at the two larger sizes, and has **lower clean BPB** and **lower degradation under every corruption** at every size — consistent with byte-level parameter sharing.

4.2 Boundary source matters

Segmentation	Boundary source	Test BPB
Fixed- p byte ($p=6$)	Every 6 bytes	1.0283
Fixed- p byte ($p=4$)	Every 4 bytes	0.9764
SpaceByte-style	Spacelike UTF-8 bytes	0.9175
BPE-guided	BPE merge table	0.8823

With the interface fixed, the **corpus statistics already in the merge table** beat fixed-length patches and hand-written surface heuristics.

4.3 Multilingual transfer

Paired $d=3072$ models trained on a ten-language FineWeb-2 subset (≈ 51 GB) with its own 128K multilingual BPE tokenizer, evaluated per language on **FLORES-200**.

Language (script)	Std BPE	BBT	$\Delta\%$
<i>Training languages</i>			
German (Latin)	1.1246	1.1097	−1.3
French (Latin)	1.0326	1.0094	−2.2
Russian (Cyrillic)	0.7010	0.6817	−2.7
Greek (Greek)	0.6512	0.6447	−1.0
Arabic (Arabic)	0.9171	0.8821	−3.8
Hindi (Devanagari)	0.6118	0.5639	−7.8
Thai (Thai)	0.5391	0.4899	−9.1
Chinese (Han)	1.4963	1.5076	+0.8
Japanese (Han+Kana)	1.0829	1.0546	−2.6
Korean (Hangul)	1.1835	1.1597	−2.0
macro-avg	0.9340	0.9104	−2.5
<i>Unseen languages (scripts seen in training)</i>			
English (Latin)	1.5364	1.4514	−5.5
Spanish (Latin)	2.0608	1.7889	−13.2
Ukrainian (Cyrillic)	1.5139	1.4057	−7.1
Bulgarian (Cyrillic)	1.8651	1.5964	−14.4
Persian (Arabic)	2.1463	2.1439	−0.1
Marathi (Devanagari)	1.2620	1.1337	−10.2
macro-avg	1.7308	1.5867	−8.3

Byte-level composition transfers **script-level structure**.